

```
1 import javax.swing.*;
2 import javax.swing.table.DefaultTableModel;
3 import java.awt.*;
4 import java.awt.event.*;
5 import java.io.IOException;
6 import java.io.InputStream;
7 import java.time.LocalDate;
8 import java.time.Period;
9 import java.time.format.DateTimeFormatter;
10 import java.util.Collections;
11 import java.util.Comparator;
12 import java.util.List;
13 import java.util.Random;
14 import org.apache.pdfbox.pdmodel.PDDocument;
15 import org.apache.pdfbox.pdmodel.PDPage;
16 import org.apache.pdfbox.pdmodel.PDPageContentStream;
17 import org.apache.pdfbox.pdmodel.common.PDRectangle;
18 import org.apache.pdfbox.pdmodel.font.PDType0Font;
19 import org.apache.pdfbox.pdmodel.graphics.color.PDColor;
20 import org.apache.pdfbox.pdmodel.graphics.color.PDDeviceRGB;
21
22 // Main application class for managing Person records
23 public class BirthdayCardGenerator extends JFrame {
24
25     // Instance variables
26     private PersonDatabase database; // Database to store persons
27     private JTextField firstNameField; // Input field for first name
28     private JTextField lastNameField; // Input field for last name
29     private JComboBox<Integer> dayComboBox; // ComboBox for day selection
```

1.1 of 13

```

30 private JComboBox<String> monthComboBox; // ComboBox for month selection
31 private JComboBox<Integer> yearComboBox; // ComboBox for year selection
32 private JComboBox<String> citizenshipComboBox; // ComboBox for citizenship selection
33 private JTable table; // Table to display persons
34 private DefaultTableModel tableModel; // Table model to manage table data
35 private static final String FONT_FILE = "Arial Bold.ttf"; // Font file for PDF generation
36
37 // Constructor to initialize the application
38 public BirthdayCardGenerator() {
39     database = new PersonDatabase(); // Initialize the database
40
41     setTitle("Birthday Card Generator"); // Set window title
42     setSize(700, 400); // Set initial window size
43     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close operation on close button
44     setLocationRelativeTo(null); // Center the window on screen
45
46     JPanel panel = new JPanel(); // Main panel for UI components
47     panel.setLayout(new GridLayout(6, 2)); // Set layout to a grid with 6 rows and 2 columns
48
49     // Add components to the panel
50     panel.add(new JLabel("First Name:"));
51     firstNameField = new JTextField();
52     panel.add(firstNameField);
53
54     panel.add(new JLabel("Last Name:"));
55     lastNameField = new JTextField();
56     panel.add(lastNameField);
57
58     panel.add(new JLabel("Date of Birth:"));

```

2.1 of 13

```

59     JPanel dobPanel = new JPanel(new FlowLayout());
60     dayComboBox = new JComboBox<>();
61     for (int i = 1; i <= 31; i++) {
62         dayComboBox.addItem(i);
63     }
64     dobPanel.add(dayComboBox);
65
66     monthComboBox = new JComboBox<>(new String[]{"January", "February", "March", "April", "May", "June",
"July", "August", "September", "October", "November", "December"});
67     dobPanel.add(monthComboBox);
68
69     yearComboBox = new JComboBox<>();
70     for (int i = 2000; i <= 2022; i++) {
71         yearComboBox.addItem(i);
72     }
73     dobPanel.add(yearComboBox);
74
75     panel.add(dobPanel);
76
77     panel.add(new JLabel("Citizenship:"));
78     citizenshipComboBox = new JComboBox<>(new String[]{
79         "Afghanistan", "Albania", "Algeria", "Andorra", "Angola", "Antigua and Barbuda",
80         "Argentina", "Armenia", "Australia", "Austria", "Azerbaijan", "Bahamas", "Bahrain",
81         "Bangladesh", "Barbados", "Belarus", "Belgium", "Belize", "Benin", "Bhutan",
82         "Bolivia", "Bosnia and Herzegovina", "Botswana", "Brazil", "Brunei", "Bulgaria",
83         "Burkina Faso", "Burundi", "Cabo Verde", "Cambodia", "Cameroon", "Canada",
84         "Central African Republic", "Chad", "Chile", "China", "Colombia", "Comoros", "Congo",
85         "Costa Rica", "Croatia", "Cuba", "Cyprus", "Czech Republic", "Denmark", "Djibouti",
86         "Dominica", "Dominican Republic", "Ecuador", "Egypt", "El Salvador", "Equatorial Guinea",

```

3.1 of 13

```

87      "Eritrea", "Estonia", "Eswatini", "Ethiopia", "Fiji", "Finland", "France", "Gabon", "Gambia",
88      "Georgia", "Germany", "Ghana", "Greece", "Grenada", "Guatemala", "Guinea", "Guinea-Bissau",
89      "Guyana", "Haiti", "Honduras", "Hungary", "Iceland", "India", "Indonesia", "Iran",
90      "Iraq", "Ireland", "Israel", "Italy", "Jamaica", "Japan", "Jordan", "Kazakhstan", "Kenya",
91      "Kiribati", "Korea, North", "Korea, South", "Kosovo", "Kuwait", "Kyrgyzstan", "Laos",
92      "Latvia", "Lebanon", "Lesotho", "Liberia", "Libya", "Liechtenstein", "Lithuania",
93      "Luxembourg", "Madagascar", "Malawi", "Malaysia", "Maldives", "Mali", "Malta", "Marshall Islands",
94      "Mauritania", "Mauritius", "Mexico", "Micronesia", "Moldova", "Monaco", "Mongolia", "Montenegro",
95      "Morocco", "Mozambique", "Myanmar", "Namibia", "Nauru", "Nepal", "Netherlands", "New Zealand",
96      "Nicaragua", "Niger", "Nigeria", "North Macedonia", "Norway", "Oman", "Pakistan", "Palau",
97      "Palestine", "Panama", "Papua New Guinea", "Paraguay", "Peru", "Philippines", "Poland",
98      "Portugal", "Qatar", "Romania", "Russia", "Rwanda", "Saint Kitts and Nevis", "Saint Lucia",
99      "Saint Vincent and the Grenadines", "Samoa", "San Marino", "Sao Tome and Principe",
100     "Saudi Arabia", "Senegal", "Serbia", "Seychelles", "Sierra Leone", "Singapore", "Slovakia",
101     "Slovenia", "Solomon Islands", "Somalia", "South Africa", "South Sudan", "Spain", "Sri Lanka",
102     "Sudan", "Suriname", "Sweden", "Switzerland", "Syria", "Taiwan", "Tajikistan", "Tanzania",
103     "Thailand", "Timor-Leste", "Togo", "Tonga", "Trinidad and Tobago", "Tunisia", "Turkey",
104     "Turkmenistan", "Tuvalu", "Uganda", "Ukraine", "United Arab Emirates", "United Kingdom",
105     "United States", "Uruguay", "Uzbekistan", "Vanuatu", "Vatican City", "Venezuela", "Vietnam",
106     "Yemen", "Zambia", "Zimbabwe"
107 }); // Populate citizenship ComboBox
108
109 panel.add(citizenshipComboBox);
110
111 JPanel buttonPanel = new JPanel(); // Panel for buttons
112 buttonPanel.setLayout(new GridLayout(1, 5)); // 1 row, 5 columns
113
114 // Create buttons and add to buttonPanel
115 JButton addButton = new JButton("Add");

```

4.1 of 13

```

116     JButton displayButton = new JButton("Display");
117     JButton searchButton = new JButton("Search");
118     JButton sortButton = new JButton("Sort");
119     JButton generateCard = new JButton("Generate Card");
120
121     buttonPanel.add(addButton);
122     buttonPanel.add(displayButton);
123     buttonPanel.add(searchButton);
124     buttonPanel.add(sortButton);
125     buttonPanel.add(generateCard);
126
127     // Add panels to JFrame using BorderLayout
128     add(panel, BorderLayout.NORTH);
129     add(buttonPanel, BorderLayout.CENTER);
130
131     // Create table and table model
132     tableModel = new DefaultTableModel(new String[]{"First Name", "Last Name", "DOB", "Citizenship", "Age"},
133 0);
134     table = new JTable(tableModel);
135     add(new JScrollPane(table), BorderLayout.SOUTH);
136
137     // Action listeners for buttons
138     addButton.addActionListener(new ActionListener() {
139         @Override
140         public void actionPerformed(ActionEvent e) {
141             addPerson();
142         }
143     });

```

5.1 of 13

```
144     displayButton.addActionListener(new ActionListener() {
145         @Override
146         public void actionPerformed(ActionEvent e) {
147             displayPersons();
148         }
149     });
150
151     searchButton.addActionListener(new ActionListener() {
152         @Override
153         public void actionPerformed(ActionEvent e) {
154             searchPerson();
155         }
156     });
157
158     sortButton.addActionListener(new ActionListener() {
159         @Override
160         public void actionPerformed(ActionEvent e) {
161             sortPersons();
162         }
163     });
164
165     generateCard.addActionListener(new ActionListener() {
166         @Override
167         public void actionPerformed(ActionEvent e) {
168             generateCard();
169         }
170     });
171
172     // Mouse listener for table (right-click to delete)
```

6.1 of 13

```

173     table.addMouseListener(new MouseAdapter() {
174         public void mousePressed(MouseEvent me) {
175             if (SwingUtilities.isRightMouseButton(me)) {
176                 int row = table.rowAtPoint(me.getPoint());
177                 if (row >= 0 && row < table.getRowCount()) {
178                     table.setRowSelectionInterval(row, row);
179                     JPopupMenu popup = new JPopupMenu();
180                     JMenuItem deleteItem = new JMenuItem("Delete");
181                     deleteItem.addActionListener(new ActionListener() {
182                         @Override
183                         public void actionPerformed(ActionEvent ev) {
184                             int selectedRow = table.getSelectedRow();
185                             if (selectedRow != -1) {
186                                 String firstName = (String) tableModel.getValueAt(selectedRow, 0);
187                                 String lastName = (String) tableModel.getValueAt(selectedRow, 1);
188                                 Person person = database.searchPerson(firstName);
189                                 if (person != null && person.getLastName().equals(lastName)) {
190                                     database.deletePerson(person);
191                                     tableModel.removeRow(selectedRow);
192                                 }
193                             }
194                         }
195                     });
196                     popup.add(deleteItem);
197                     popup.show(me.getComponent(), me.getX(), me.getY());
198                 }
199             }
200         }
201     });

```

7.1 of 13

```

202     }
203
204     // Method to add a new person to the database and update the table
205     private void addPerson() {
206         String firstName = firstNameField.getText();
207         String lastName = lastNameField.getText();
208         String dob = String.format("%04d-%02d-%02d", (int) yearComboBox.getSelectedItem(),
monthComboBox.getSelectedIndex() + 1, (int) dayComboBox.getSelectedIndex());
209         String citizenship = (String) citizenshipComboBox.getSelectedItem();
210         if (!(firstName.equals("") || lastName.equals("")) { // Check if first name or last name is not empty
211             database.addPerson(new Person(firstName, lastName, dob, citizenship)); // Add person to database
212             displayPersons(); // Refresh table display
213         } else {
214             JOptionPane.showMessageDialog(null, "First name or Last name cannot be left blank"); // Show error
message if first name or last name is blank
215         }
216     }
217
218     // Method to display all persons in the database in the table
219     private void displayPersons() {
220         List<Person> persons = database.getPersons(); // Retrieve all persons from database
221         tableModel.setRowCount(0); // Clear existing table rows
222         for (Person person : persons) { // Iterate through each person and add to table model
223             tableModel.addRow(new Object[]{person.getFirstName(), person.getLastName(), person.getDateOfBirth(),
person.getCitizenship(), calculateAge(person.getDateOfBirth())});
224         }
225     }
226
227     // Method to search for persons by first name and display results in the table

```

8.1 of 13


```

228     private void searchPerson() {
229         String firstName = JOptionPane.showInputDialog(this, "Enter first name to search:"); // Prompt user for
first name input
230         if (firstName != null && !firstName.isEmpty()) { // Check if first name input is not null or empty
231             tableModel.setRowCount(0); // Clear existing table rows
232
233             List<Person> persons = database.getPersons(); // Retrieve all persons from database
234             for (Person person : persons) { // Iterate through each person
235                 if (person.getFirstName().toLowerCase().startsWith(firstName.toLowerCase())) { // Check if first
name matches search input (case insensitive)
236                     tableModel.addRow(new Object[]{person.getFirstName(), person.getLastName(),
person.getDateOfBirth(), person.getCitizenship(), calculateAge(person.getDateOfBirth())}); // Add matching person to
table model
237                 }
238             }
239
240             if (tableModel.getRowCount() == 0) { // If no matching records found, display message
241                 JOptionPane.showMessageDialog(this, "No records found for " + firstName);
242             }
243         }
244     }
245
246     // Method to sort persons alphabetically by first name and update the table
247     private void sortPersons() {
248         List<Person> persons = database.getPersons(); // Retrieve all persons from database
249
250         // Sort persons list alphabetically by First Name
251         Collections.sort(persons, new Comparator<Person>() {
252             @Override

```

9.1 of 13

```

253         public int compare(Person p1, Person p2) {
254             return p1.getFirstName().compareTo(p2.getFirstName());
255         }
256     });
257
258     // Clear the table before adding sorted data
259     tableModel.setRowCount(0);
260
261     // Add sorted data to the table model
262     for (Person person : persons) {
263         tableModel.addRow(new Object[]{person.getFirstName(), person.getLastName(), person.getDateOfBirth(),
person.getCitizenship(), calculateAge(person.getDateOfBirth())});
264     }
265 }
266
267 // Method to generate a birthday card PDF for the selected person
268 private void generateCard() {
269     int selectedRow = table.getSelectedRow(); // Get selected row from table
270     if (selectedRow == -1) { // If no row is selected, show error message
271         JOptionPane.showMessageDialog(this, "Please select a person from the table.", "Error",
JOptionPane.ERROR_MESSAGE);
272         return;
273     }
274
275     String selectedName = (String) table.getValueAt(selectedRow, 0); // Get selected person's first name
276
277     try {
278         PDDocument document = new PDDocument(); // Create a new PDF document
279         PDPage page = new PDPage(PDRectangle.A4); // Create a new page (A4 size)

```

10.1 of 13

```

280     document.addPage(page); // Add page to document
281
282     // Load custom TTF font
283     InputStream fontStream = getClass().getResourceAsStream(FONT_FILE); // Load font file
284     if (fontStream == null) {
285         throw new IOException("Font file not found: " + FONT_FILE); // Throw exception if font file not
found
286     }
287     PDType0Font font = PDType0Font.load(document, fontStream); // Load font
288
289     PDPageContentStream contentStream = new PDPageContentStream(document, page); // Create content
stream for drawing
290
291     // Draw red border around card
292     PDColor red = new PDColor(new float[]{1, 0, 0}, PDDeviceRGB.INSTANCE); // Define red color
293     contentStream.setStrokingColor(red); // Set stroking color (border color)
294     contentStream.setLineWidth(5); // Set border line width
295     contentStream.addRect(50, 600, 500, 200); // Draw rectangle (x, y, width, height)
296     contentStream.stroke(); // Stroke (draw) the rectangle
297
298     // Set font and font size for text
299     contentStream.setFont(font, 20); // Set font and size
300
301     // Draw birthday message
302     contentStream.beginText(); // Begin text mode for drawing text
303     contentStream.newLineAtOffset(100, 700); // Set text position (x, y)
304     contentStream.showText("Happy Birthday to " + selectedName + "!"); // Display birthday message
305     contentStream.endText(); // End text mode
306

```

11.1 of 13

```

307         Random rnd=new Random();
308         String [] message={
309             "Best wishes for a fantastic day!",
310             "Wishing you all the best today",
311             "It is your day to shine"
312         };
313         int index=rnd.nextInt(3);
314
315         // Draw additional content (example)
316         contentStream.beginText(); // Begin new text mode
317         contentStream.newLineAtOffset(100, 680); // Set new text position
318         contentStream.showText(message[index]); // Display additional message
319         contentStream.endText(); // End text mode
320
321         contentStream.close(); // Close content stream
322
323         // Save document to file
324         String fileName = selectedName + "_birthday_card.pdf"; // Generate file name
325         document.save(fileName); // Save PDF document
326         document.close(); // Close the document
327
328         JOptionPane.showMessageDialog(this, "Birthday card generated successfully: " + fileName); // Show
success message
329     } catch (IOException ex) {
330         JOptionPane.showMessageDialog(this, "Error generating birthday card: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE); // Show error message
331     }
332 }
333

```

12.1 of 13

```
334 // Method to calculate age based on date of birth
335 private int calculateAge(String dob) {
336     DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
337     LocalDate birthDate = LocalDate.parse(dob, formatter);
338     LocalDate currentDate = LocalDate.now();
339     return Period.between(birthDate, currentDate).getYears();
340 }
341
342 // Main method to start the application
343 public static void main(String[] args) {
344     SwingUtilities.invokeLater(new Runnable() {
345         @Override
346         public void run() {
347             new BirthdayCardGenerator().setVisible(true); // Create and display the application window
348         }
349     });
350 }
351 }
```

13.1 of 13

```
1 import java.io.Serializable;
2
3 // A Serializable class representing a person
4 public class Person implements Serializable {
5
6     // Instance variables
7     private String firstName;
8     private String lastName;
9     private String dateOfBirth;
10    private String citizenship;
11
12    // Constructor to initialize a Person object
13    public Person(String firstName, String lastName, String dateOfBirth, String citizenship) {
14        this.firstName = firstName;
15        this.lastName = lastName;
16        this.dateOfBirth = dateOfBirth;
17        this.citizenship = citizenship;
18    }
19
20    // Getter method for firstName
21    public String getFirstName() {
22        return firstName;
23    }
24
25    // Getter method for lastName
26    public String getLastName() {
27        return lastName;
28    }
29
```

1.1 of 2

```
30    // Getter method for dateOfBirth
31    public String getDateOfBirth() {
32        return dateOfBirth;
33    }
34
35    // Getter method for citizenship
36    public String getCitizenship() {
37        return citizenship;
38    }
39
40    // Override toString method to provide a string representation of the Person object
41    @Override
42    public String toString() {
43        return firstName + " " + lastName + " " + dateOfBirth + " " + citizenship;
44    }
45 }
```

2.1 of 2

```

1 import java.io.*;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 // Class to manage a database of Person records, stored in a text file
6 public class PersonDatabase {
7     // File name where persons data is stored
8     private static final String FILE_NAME = "persons.txt";
9     private List<Person> persons; // List to store Person objects
10
11     // Constructor to initialize the database and load data from file
12     public PersonDatabase() {
13         persons = new ArrayList<>(); // Initialize the list of persons
14         loadFromFile(); // Load persons data from file into the list
15     }
16
17     // Method to add a new person to the database and save to file
18     public void addPerson(Person person) {
19         persons.add(person); // Add person to the list
20         saveToFile(); // Save updated list to file
21     }
22
23     // Method to retrieve a copy of all persons in the database
24     public List<Person> getPersons() {
25         return new ArrayList<>(persons); // Return a copy of the list to prevent direct modification
26     }
27
28     // Method to search for a person by first name or last name
29     public Person searchPerson(String name) {

```

1.1 of 3


```

30     for (Person person : persons) {
31         // Case-insensitive check if the name matches first name or last name
32         if (person.getFirstName().equalsIgnoreCase(name) || person.getLastName().equalsIgnoreCase(name)) {
33             return person; // Return the person if found
34         }
35     }
36     return null; // Return null if person not found
37 }
38
39 // Method to delete a person from the database and save to file
40 public void deletePerson(Person person) {
41     persons.remove(person); // Remove person from the list
42     saveToFile(); // Save updated list to file
43 }
44
45 // Private method to load persons data from file into the list
46 private void loadFromFile() {
47     try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
48         String line;
49         while ((line = reader.readLine()) != null) {
50             String[] parts = line.split(","); // Split line by comma
51             if (parts.length == 4) { // Ensure line has correct number of parts
52                 String firstName = parts[0];
53                 String lastName = parts[1];
54                 String dateOfBirth = parts[2];
55                 String citizenship = parts[3];
56                 persons.add(new Person(firstName, lastName, dateOfBirth, citizenship)); // Create Person
57             }
58         }
59     }
60 }

```

2.1 of 3

```
58         }
59     } catch (IOException e) {
60         e.printStackTrace(); // Print stack trace if file reading fails
61     }
62 }
63
64 // Private method to save persons data from list to file
65 private void saveToFile() {
66     try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_NAME))) {
67         for (Person person : persons) {
68             // Write person details separated by commas and newline
69             writer.write(String.join(",", person.getFirstName(), person.getLastName(),
person.getDateOfBirth(), person.getCitizenship()));
70             writer.newLine();
71         }
72     } catch (IOException e) {
73         e.printStackTrace(); // Print stack trace if file writing fails
74     }
75 }
76 }
```

3.1 of 3